

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('cleaned_reviews_data.csv')
```

```
df.head()
```

	Unnamed: 0	Ratings	Reviews	movie_name	Resenhas	genres	Description	emotion	clean_description	clean_review
0	0	3	It had some laughs, but overall the motivation...	Waiting to Exhale	Riu algumas risadas, mas no geral a motivação...	['Comedy', 'Drama', 'Romance']	Based on Terry McMillan's novel, this film fol...	anticipation	based on terry mcmillans novel this film follo...	it had sor laughs b overall t motivation
1	1	4	WAITING TO EXHALE Waiting, and waiting, and wa...	Waiting to Exhale	ESPERANDO PARA EXALAR Esperando, e esperando, ...	['Comedy', 'Drama', 'Romance']	Based on Terry McMillan's novel, this film fol...	anticipation	based on terry mcmillans novel this film follo...	waiting exhale waiti and waiting a wai
			Angela		Angela		Based on			

```
df.columns
```

```
Index(['Unnamed: 0', 'Ratings', 'Reviews', 'movie_name', 'Resenhas', 'genres',
      'Description', 'emotion', 'clean_description', 'clean_reviews'],
      dtype='object')
```

```
print(df.shape)
```

```
(46173, 10)
```

```
print(df[['movie_name', 'genres', 'Ratings']].isnull().sum())
```

```
movie_name    0
genres        0
Ratings       0
dtype: int64
```

```
df['genres'].head(10)
```

	genres
0	['Comedy', 'Drama', 'Romance']
1	['Comedy', 'Drama', 'Romance']
2	['Comedy', 'Drama', 'Romance']
3	['Comedy', 'Drama', 'Romance']
4	['Comedy', 'Drama', 'Romance']
5	['Comedy', 'Drama', 'Romance']
6	['Comedy', 'Drama', 'Romance']
7	['Comedy', 'Drama', 'Romance']
8	['Comedy']
9	['Comedy']

```
dtype: object
```

```
genre_df = df.copy()
```

```
genre_df['genres'] = genre_df['genres'].str.replace('[', '', regex=False)
```

```
genre_df['genres'] = genre_df['genres'].str.replace(']', '', regex=False)
```

```
genre_df['genres'] = genre_df['genres'].str.replace('"', '', regex=False)
```

```
genre_df['genres'] = genre_df['genres'].str.split(',')
```

```
genre_df = genre_df.explode('genres')
```

```
genre_df['genres'] = genre_df['genres'].str.strip()
```

```
print(genre_df[['movie_name', 'genres']].head(20))
```

```
   movie_name  genres
0  Waiting to Exhale  Comedy
0  Waiting to Exhale   Drama
0  Waiting to Exhale  Romance
1  Waiting to Exhale  Comedy
1  Waiting to Exhale   Drama
1  Waiting to Exhale  Romance
2  Waiting to Exhale  Comedy
2  Waiting to Exhale   Drama
2  Waiting to Exhale  Romance
3  Waiting to Exhale  Comedy
3  Waiting to Exhale   Drama
3  Waiting to Exhale  Romance
4  Waiting to Exhale  Comedy
4  Waiting to Exhale   Drama
4  Waiting to Exhale  Romance
5  Waiting to Exhale  Comedy
5  Waiting to Exhale   Drama
5  Waiting to Exhale  Romance
6  Waiting to Exhale  Comedy
6  Waiting to Exhale   Drama
```

```
genre_review_count = genre_df.groupby('genres').size()
genre_review_count = genre_review_count.reset_index()
genre_review_count.columns = ['genre', 'review_count']
```

```
genre_review_count = genre_review_count.sort_values(
    by='review_count',
    ascending=False
)
```

```
genre_review_count.head(10)
```

	genre	review_count
13	Drama	23128
24	Romance	19156
10	Comedy	15872
28	Thriller	8586
6	Action	6792
15	Fantasy	5555
18	Horror	4876
14	Family	4875
8	Adventure	4747
11	Crime	3980

```
genre_median = genre_review_count['review_count'].median()
```

```
print("Genre Review Count Median:", genre_median)
```

```
Genre Review Count Median: 6.0
```

```
high_engagement_genres = genre_review_count[
    genre_review_count['review_count'] > genre_median
]
```

```
high_engagement_genres
```

	genre	review_count
13	Drama	23128
24	Romance	19156
10	Comedy	15872
28	Thriller	8586
6	Action	6792
15	Fantasy	5555
18	Horror	4876
14	Family	4875
8	Adventure	4747
11	Crime	3980
25	Science Fiction	3268
21	Mystery	2987
20	Music	2189
9	Animation	1648
16	Foreign	1063
17	History	1003
12	Documentary	912
0		880
29	War	595
26	TV Movie	536
30	Western	426
35	essa estr�cia direcional de Joel Egerton � um...	9

```

selected_genres = high_engagement_genres['genre'].tolist()

filtered_genre_df = genre_df[
    genre_df['genres'].isin(selected_genres)
]

genre_avg_rating = filtered_genre_df.groupby(
    'genres'
)['Ratings'].mean()

genre_avg_rating = genre_avg_rating.reset_index()

genre_avg_rating.columns = [
    'genre',
    'average_rating'
]

genre_avg_rating = genre_avg_rating.sort_values(
    by='average_rating',
    ascending=False
)

print(genre_avg_rating)

```

	genre	average_rating
21	essa estr�cia direcional de Joel Egerton � um...	7.000000
11	History	6.715852
6	Documentary	6.468202
5	Crime	6.276131
4	Comedy	6.162172
7	Drama	6.140393
2	Adventure	6.130819
17	TV Movie	6.115672
15	Romance	6.072144
1	Action	5.966578
18	Thriller	5.916608
14	Mystery	5.885169
3	Animation	5.837379
19	War	5.776471
13	Music	5.773869
8	Family	5.750974
16	Science Fiction	5.705018
12	Horror	5.676784
20	Western	5.584507

9	Fantasy	5.486589
10	Foreign	5.127940
0		4.919318

```
top_genre = genre_avg_rating.iloc[0]
```

```
print("Highest-Rated High-Engagement Genre:")
print(top_genre)
```

```
Highest-Rated High-Engagement Genre:
genre          essa estr ia direcional de Joel Egerton    um...
average_rating          7.0
Name: 21, dtype: object
```

crime

```
crime_df = genre_df[
    genre_df['genres'] == 'Crime'
]
```

```
total_crime_movies = crime_df['movie_name'].nunique()
```

```
print("Total Number of Crime Movies:")
print(total_crime_movies)
```

```
Total Number of Crime Movies:
192
```

```
crime_movie_names = crime_df['movie_name'].unique()
```

```
print(crime_movie_names)
```

```
['Heat' 'Taxi Driver' 'The Net' 'Exit to Eden' 'Speed' 'Bad Company'
 'True Romance' 'The Escort' 'A Thin Line Between Love and Hate'
 'Nothing to Lose' 'The Maltese Falcon' 'The Thin Man' 'Stand by Me' 'M'
 'Turbulence' 'Kicked in the Head' 'Fire Down Below' 'The Assignment'
 'Witness' 'Fallen' 'Twilight' 'Les Mis rables' 'Out of Sight'
 'West Side Story' 'Phoenix' 'Gloria' 'Payback' 'Go' 'Life'
 'Mildred Pierce' 'The Morning After' 'The Thomas Crown Affair'
 'Mickey Blue Eyes' 'Miss Congeniality' 'Panic' 'Sugar & Spice'
 'Saving Silverman' 'The Mexican' 'Beautiful Creatures'
 'The Fast and the Furious' 'The Score' 'Ariel' 'Sunset' 'Family Business'
 'Her Alibi' 'Bangkok Dangerous' 'The Man Who Wasn't There' 'The Formula'
 'The Gambler' 'Mo' Money' 'Butterfly' 'A Kiss Before Dying'
 'Confessions of a Dangerous Mind' 'Lockdown' 'The Pink Panther'
 'Paper Moon' 'Cobra' 'Intolerable Cruelty' 'The Hard Way' 'Revenge'
 'Dragnet' 'Accident' 'The Pick-up Artist' 'Hell's Kitchen' 'The Chase'
 'The Getaway' 'Rush' 'White Heat' 'Riders' 'Taxi' 'The Forgotten'
 'The Raven' 'The Letter' 'The Stranger' 'The Hound of the Baskervilles'
 'The Hunters' 'Avenging Angelo' '3-Iron' 'Siberia' 'Inferno'
 'King's Ransom' 'Unleashed' 'The Deal' 'Kiss Kiss Bang Bang' 'The Wraith'
 'Match Point' 'The Protector' 'My Super Ex-Girlfriend' 'Compulsion'
 'Step Up' 'The Good German' 'Unknown' 'Shooter' 'No Way Out' 'Nancy Drew'
 '21' 'The Girl Next Door' 'Wanted' 'Game Over' 'Night Train' 'Chocolate'
 'The Bandit' 'Public Enemies' 'Fighting' 'The Secret in Their Eyes'
 'Rampage' 'Underworld' 'Sisters' 'The Next Three Days' 'Nightfall'
 'I Spit on Your Grave' 'Henry's Crime' 'Dark City' 'Trust'
 'Happy New Year' 'J. Edgar' 'Faces in the Crowd' 'Project X' '3 A.M.'
 'Fire with Fire' 'Side Effects' 'The Call' 'Players' 'The Best Offer'
 'The Heat' 'Passion' 'Prisoners' 'The Family' 'The Factory' 'Angel'
 'You May Not Kiss the Bride' 'The Outsider' 'The Road' 'Jailbait' 'Steel'
 'Wolf' 'Rage' 'Trick' 'Gunday' 'Crossroads' 'The Captive' 'Plastic'
 'Inherent Vice' 'Hit by Lightning' 'Little Italy' 'Killers' 'The Squeeze'
 'Inside Out' 'Tempo' 'Girl in Gold Boots' 'Midnight' 'Focus' 'Rio'
 'Two of a Kind' 'Obsession' 'Mother's Day' 'Spy' 'Suicide Squad'
 'Wild Horses' 'The Teacher' 'Superman' 'Legend' 'Gone' 'Badge of Honor'
 'Blind Date' 'Aflatoon' 'Bloodline' 'Manhunt' 'Dilwale' 'Sleepless'
 'Raavan' 'Jackpot' 'Exposed' 'Ride or Die' 'Gangster'
 'The Girl in the Photographs' 'The Trust' 'War Dogs' 'The Seamstress'
 'The Neighbor' 'Snatched' 'Kajraare' 'Coin Heist' 'The Crash'
 'Jungle Goddess' 'The Girl in Lovers Lane' 'Radar Secret Service'
 'Cannibal' 'Exit' 'The Promise' 'Ecstasy' 'The Saint']
```

```
crime_movie_review_count = crime_df.groupby(
    'movie_name'
).size()

crime_movie_review_count = crime_movie_review_count.reset_index()

crime_movie_review_count.columns = [
    'movie_name',
    'review_count'
]
```

```

crime_movie_review_count = crime_movie_review_count.sort_values(
    by='review_count',
    ascending=False
)

print(crime_movie_review_count)

```

	movie_name	review_count
45	Happy New Year	266
122	Taxi	184
160	The Promise	154
14	Blind Date	152
101	Revenge	130
..	...	...
6	Aflatoon	1
8	Ariel	1
69	Lockdown	1
97	Raavan	1
153	The Morning After	1

[192 rows x 2 columns]

```

high_engagement_crime_movies = crime_movie_review_count[

    crime_movie_review_count['review_count']
    > 14
]

print(high_engagement_crime_movies)

```

	movie_name	review_count
45	Happy New Year	266
122	Taxi	184
160	The Promise	154
14	Blind Date	152
101	Revenge	130
..	...	...
173	The Trust	15
180	Two of a Kind	15
127	The Best Offer	15
109	Side Effects	15
13	Beautiful Creatures	15

[95 rows x 2 columns]

```

crime_df = genre_df[
    genre_df['genres'] == 'Crime'
]

print(crime_df.head())

```

	Unnamed: 0	Ratings	Reviews
26	26	4	Take the big moment of this movie. What the tr...
27	27	4	What passes for epic cinema these days can be ...
28	28	6	Having seen almost every classic and influenti...
29	29	9	Micheal Manns'(director of 'The Insider', and ...
241	241	2	Reading the above comments just makes me reach...

	movie_name	Resenhas	genres
26	Heat	Tome o grande momento deste filme. O que os tr...	Crime
27	Heat	O que passa pelo cinema Ã©pico nos dias de hoj...	Crime
28	Heat	Tendo visto quase todos os drama policial clÃ¡...	Crime
29	Heat	Micheal Manns (diretor de 'The Insider' e 'Man...	Crime
241	Taxi Driver	Ler os comentÃ¡rios acima apenas me faz alcanÃ...	Crime

	Description	emotion
26	Spoof of the casual sexual adventures of a one...	disgust
27	Spoof of the casual sexual adventures of a one...	disgust
28	Spoof of the casual sexual adventures of a one...	disgust
29	Spoof of the casual sexual adventures of a one...	disgust
241	Mangal drives a taxi by day, then drinks at ni...	sadness

	clean_description
26	spoof of the casual sexual adventures of a one...
27	spoof of the casual sexual adventures of a one...
28	spoof of the casual sexual adventures of a one...
29	spoof of the casual sexual adventures of a one...
241	mangal drives a taxi by day then drinks at nig...

	clean_reviews
26	take the big moment of this movie what the tra...
27	what passes for epic cinema these days can be ...
28	having seen almost every classic and influenti...
29	micheal mannsdirector of the insider and manhu...
241	reading the above comments just makes me reach...

```

crime_movie_review_count = crime_df.groupby(
    'movie_name'
).size()

crime_movie_review_count = crime_movie_review_count.reset_index()

crime_movie_review_count.columns = [
    'movie_name',
    'review_count'
]

```

```

crime_movie_median = crime_movie_review_count[
    'review_count'
].median()

print(crime_movie_median)

```

14.0

```

high_engagement_crime_movies = crime_movie_review_count[

    crime_movie_review_count['review_count']
    > crime_movie_median
]

```

```

crime_movie_names = high_engagement_crime_movies[
    'movie_name'
].tolist()

```

```

filtered_crime_df = crime_df[
    crime_df['movie_name'].isin(crime_movie_names)
]

```

```

crime_movie_avg_rating = filtered_crime_df.groupby(
    'movie_name'
)['Ratings'].mean()

crime_movie_avg_rating = crime_movie_avg_rating.reset_index()

crime_movie_avg_rating.columns = [
    'movie_name',
    'average_rating'
]

crime_movie_avg_rating = crime_movie_avg_rating.sort_values(

    by='average_rating',

    ascending=False
)

```

```

top_10_crime_movies = crime_movie_avg_rating.head(10)

top_10_crime_movies = top_10_crime_movies.reset_index(
    drop=True
)

print(top_10_crime_movies)

```

	movie_name	average_rating
0	The Promise	9.590909
1	The Girl Next Door	9.214286
2	Rampage	8.904762
3	The Fast and the Furious	8.666667
4	Happy New Year	8.631579
5	My Super Ex-Girlfriend	8.166667
6	The Thomas Crown Affair	8.166667
7	Les Misérables	8.100000
8	White Heat	8.000000
9	Wanted	7.944444

```

bottom_10_crime_movies = crime_movie_avg_rating.tail(10)

bottom_10_crime_movies = bottom_10_crime_movies.reset_index(
    drop=True
)

print(bottom_10_crime_movies)

```

	movie_name	average_rating
0	The Hunters	4.541667
1	Siberia	4.454545
2	The Deal	4.400000
3	Wild Horses	4.277778
4	The Raven	3.272727
5	The Heat	3.240000
6	Crossroads	3.105263
7	Gunday	2.857143
8	Girl in Gold Boots	2.352941
9	Nightfall	1.888889

```
top_10_crime_movie_names = top_10_crime_movies[
    'movie_name'
].tolist()

bottom_10_crime_movie_names = bottom_10_crime_movies[
    'movie_name'
].tolist()
```

```
print(top_10_crime_movie_names)
```

```
print(bottom_10_crime_movie_names)
```

```
['The Promise', 'The Girl Next Door', 'Rampage', 'The Fast and the Furious', 'Happy New Year', 'My Super Ex-Girlfriend', 'The Hunters', 'Siberia', 'The Deal', 'Wild Horses', 'The Raven', 'The Heat', 'Crossroads', 'Gunday', 'Girl in Gold Boots',
```

```
top_crime_reviews = filtered_crime_df[
    filtered_crime_df['movie_name'].isin(
        top_10_crime_movie_names
    )
]

bottom_crime_reviews = filtered_crime_df[
    filtered_crime_df['movie_name'].isin(
        bottom_10_crime_movie_names
    )
]

print(top_crime_reviews.shape)

print(bottom_crime_reviews.shape)
```

```
(681, 10)
(304, 10)
```

```
top_10_movies = [
    'The Promise',
    'The Girl Next Door',
    'Rampage',
    'The Fast and the Furious',
    'Happy New Year',
    'My Super Ex-Girlfriend',
    'The Thomas Crown Affair',
    'Les Misérables',
    'White Heat',
    'Wanted'
]

bottom_10_movies = [
    'The Hunters',
    'Siberia',
    'The Deal',
    'Wild Horses',
    'The Raven',
    'The Heat',
    'Crossroads',
    'Gunday',
    'Girl in Gold Boots',
    'Nightfall'
]
```

```
top_10_df = df[
```

```
df['movie_name'].isin(top_10_movies)
]
bottom_10_df = df[

df['movie_name'].isin(bottom_10_movies)
]
```

```
print(top_10_df[['movie_name']].drop_duplicates())
print(bottom_10_df[['movie_name']].drop_duplicates())
```

```

movie_name
1468      The Promise
3867      Les Misérables
5546      The Thomas Crown Affair
7937      The Fast and the Furious
12126     The Girl Next Door
12820     White Heat
17059     My Super Ex-Girlfriend
19312     Wanted
22090     Rampage
25126     Happy New Year
movie_name
5631      The Raven
8910      Crossroads
13066     The Hunters
14920     Siberia
15177     Wild Horses
15549     The Deal
23538     Nightfall
28276     The Heat
31489     Gunday
34422     Girl in Gold Boots
```

```
!pip install google-genai pandas openpyxl pydantic
```

```

Requirement already satisfied: google-genai in /usr/local/lib/python3.12/dist-packages (1.68.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: openpyxl in /usr/local/lib/python3.12/dist-packages (3.1.5)
Requirement already satisfied: pydantic in /usr/local/lib/python3.12/dist-packages (2.12.3)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.13.0)
Requirement already satisfied: google-auth<3.0.0,>=2.47.0 in /usr/local/lib/python3.12/dist-packages (from google-auth[request)] (2.40.0)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (0.28.1)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai) (2.32.3)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai) (9.1.4)
Requirement already satisfied: websockets<17.0,>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (15.0.7)
Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (4.12.2)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.9.0)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-packages (from openpyxl) (2.0.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.4.2)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-genai) (3.10.1)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.47.0->google-auth[request)] (0.6.1)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.47.0->google-auth[request)] (4.9.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (2025.11.11)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-genai) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>=0.28.1->google-genai) (0.14.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (3.4.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1->google-genai) (2.3.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.47.0->google-auth[request)] (0.6.1)
```

```

import os
from google import genai

os.environ["GEMINI_API_KEY"] = "AIzaSyBbDuN-oxUPDxy84pIDZjmvAX2XGjok3AHw"

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
```

```

import pandas as pd

file_path = "/content/top_10_crime_movies.xlsx"

df = pd.read_excel(file_path)

print(df.columns)
df.head()
```



```
Index(['Unnamed: 0', 'Ratings', 'Reviews', 'movie_name', 'Resenhas', 'genres',
      'Description', 'emotion', 'clean_description', 'clean_reviews'],
      dtype='object')
```

	Unnamed: 0	Ratings	Reviews	movie_name	Resenhas	genres	Description	emotion	clean_description	clean_reviews
0	1468	7	What madness is this? I just arrived from cine...	The Promise	Que loucura Ã© essa? Acabei de chegar do cinem...	['Drama', 'Romance']	A corporate heir's mother pays for his disfigu...	anger	a corporate heirs mother pays for his disfigur...	what madness is this i just arrived from cinem...
1	1469	7	What madness is this? I just arrived from	The Promise	Que loucura Ã© essa? Acabei de chegar do	['Drama', 'Romance']	Daniel and Andrea grew up in the same home and...	sadness	daniel and andrea grew up in the same home and...	what madness is this i just arrived from cinem...

```
reviews = df["Reviews"].dropna().astype(str).drop_duplicates().tolist()

print(len(reviews))
```

168

```
from pydantic import BaseModel, Field
from typing import List, Literal

class KeywordPhrase(BaseModel):
    phrase: str = Field(description="A useful content-related keyword phrase, preferably 2-4 words.")

    category: Literal[
        "genre_element",
        "story_element",
        "pacing",
        "suspense",
        "character",
        "acting",
        "realism",
        "emotion",
        "production",
        "audience_engagement",
        "hybrid_genre"
    ]

    reason: str = Field(
        description="Why this phrase is useful for content planning."
    )

class KeywordResult(BaseModel):
    phrases: List[KeywordPhrase]

def extract_useful_phrases_with_gemini(review_batch):

    reviews_text = "\n\n--\n\n".join(review_batch)
```

```

    prompt = f"""
    You are a professional content analyst for a streaming company.

    Task:
    Extract useful keyword phrases from the following crime movie reviews.

    Focus on phrases that can help with content planning, especially:
    - genre elements
    - story elements
    - pacing
    - suspense
    - plot structure
    - character development
    - acting / performance
    - realism
    - emotional tone
    - audience engagement
    - production quality
    - hybrid genre elements

    Avoid:
    - actor names
    - movie titles
    - country names
    - vague praise words
    - meaningless common words

    Return useful phrases, preferably 2-4 words each.

    Reviews:
    {reviews_text}
    """

```

```

    response = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt,
        config={
            "temperature": 0.2,
            "response_mime_type": "application/json",
            "response_schema": KeywordResult,
        }
    )

    return response.parsed

```

```

def chunk_list(data, chunk_size=20):
    for i in range(0, len(data), chunk_size):
        yield data[i:i + chunk_size]

review_chunks = list(chunk_list(reviews, 20))

print(len(review_chunks))

```

9

```

all_phrases = []

for i, chunk in enumerate(review_chunks[:2]):

    print(f"\nProcessing chunk {i+1}...")

    try:

        result = extract_useful_phrases_with_gemini(chunk)

        print(result)

        for item in result.phrases:

            all_phrases.append({
                "phrase": item.phrase.lower().strip(),
                "category": item.category,
                "reason": item.reason
            })

    except Exception as e:

        import traceback
        traceback.print_exc()

```

```
Processing chunk 1...
phrases=[KeywordPhrase(phrase='historical drama', category='genre_element', reason='Identifies the core genre of the film, u

Processing chunk 2...
phrases=[KeywordPhrase(phrase='cat and mouse thriller', category='hybrid_genre', reason='Identifies a specific subgenre bler
```

```
phrases_df = pd.DataFrame(all_phrases)

phrases_df.head(20)
```

	phrase	category	reason
0	historical drama	genre_element	Identifies the core genre of the film, useful ...
1	love triangle	story_element	A classic narrative device that drives charact...
2	romance subplot	story_element	Adds emotional depth and broadens audience app...
3	edited tighter	pacing	Highlights the need for concise editing to mai...
4	edge of my seat	suspense	Indicates high tension and strong audience eng...
5	vibrant characters	character	Emphasizes the importance of well-developed, m...
6	period costumes	production	Crucial for establishing historical authentici...
7	historical events	realism	Grounding the narrative in real-world history ...
8	emotionally draining	emotion	Describes the intense psychological impact of ...
9	suspense of disbelief	audience_engagement	Reflects the film's ability to immerse the vie...
10	humor and romance	hybrid_genre	Combining lighter elements with a heavy histor...
11	climax and ending	pacing	Essential structural elements that determine t...
12	cat and mouse thriller	hybrid_genre	Identifies a specific subgenre blend of suspen...
13	caper film	genre_element	Defines the specific heist/crime subgenre, hel...
14	character development	character	Crucial for assessing narrative depth and view...
15	breakneck speed	pacing	Describes a fast-paced narrative flow, useful ...
16	unconventional twists	story_element	Highlights unpredictable plot structures that ...
17	wooden acting	acting	Critical feedback on performance quality, usef...
18	cgi reconstructions	production	Identifies the visual style and reliance on di...
19	tearjerking scene	emotion	Pinpoints highly emotional, melodramatic momen...

```
all_phrases = []

for i, chunk in enumerate(review_chunks):
    print(f"Processing chunk {i+1}/{len(review_chunks)}...")

    try:
        result = extract_useful_phrases_with_gemini(chunk)

        for item in result.phrases:
            all_phrases.append({
                "phrase": item.phrase.lower().strip(),
                "category": item.category,
                "reason": item.reason
            })

    except Exception as e:
        print(f"Error in chunk {i+1}: {e}")

phrases_df = pd.DataFrame(all_phrases)

phrases_df.head()
```

```
Processing chunk 1/9...
Processing chunk 2/9...
Processing chunk 3/9...
Processing chunk 4/9...
Processing chunk 5/9...
Processing chunk 6/9...
Processing chunk 7/9...
Processing chunk 8/9...
Processing chunk 9/9...
```

	phrase	category	reason
0	historical epic	genre_element	Identifies the broad genre appeal for audience...
1	love triangle	story_element	A classic narrative trope that drives characte...
2	suspense and danger	suspense	Highlights the tension-building elements cruci...
3	never feels rushed	pacing	Indicates a well-balanced narrative flow that ...
4	period costumes	production	Key production design element that enhances vi...

```
summary_df = (
    phrases_df
    .groupby(["phrase", "category"])
    .agg(
        count=("phrase", "size"),
        reason=("reason", "first")
    )
    .reset_index()
    .sort_values(by="count", ascending=False)
)

summary_df.head(30)
```

	phrase	category	count	reason
11	character development	character	3	Crucial for assessing how deeply the audience ...
2	adrenaline rush	audience_engagement	2	Captures the physical and emotional impact of ...
33	fast-paced action	pacing	2	Describes the high-tempo narrative flow and ed...
107	undercover cop	character	2	A classic crime archetype that introduces inhe...
70	on-screen chemistry	acting	2	Essential for romantic crime thrillers where t...
78	roller coaster ride	pacing	2	Describes high-energy, fast-paced narrative pr...
6	breakneck speed	pacing	1	Describes a fast-paced narrative structure tha...
1	action packed scenes	genre_element	1	Identifies high-energy sequences that attract ...
4	bipolar superhero	character	1	Helps in planning unique character archetypes ...
0	action comedy	hybrid_genre	1	Represents a popular genre blend that balances...
9	cat and mouse thriller	hybrid_genre	1	Identifies a specific subgenre blend of romanc...
10	cgi reconstructions	production	1	Relates to the visual quality and historical w...
12	character transformation	character	1	Crucial for tracking protagonist development f...
13	choppy pacing	pacing	1	Identifies potential structural issues in the ...
14	coming-of-age themes	story_element	1	Captures the emotional and developmental journ...
7	caper film	genre_element	1	A specific crime subgenre focusing on a meticu...
8	caper to cover-up	story_element	1	Highlights a specific narrative progression in...
3	average joe	character	1	A highly relatable protagonist archetype that ...
17	curated soundtrack	production	1	Emphasizes the role of music in enhancing the ...
18	dark and brooding	emotion	1	Captures the emotional atmosphere and mood of ...
20	dark humor	hybrid_genre	1	Blending comedy with dark or violent themes to...
19	dark humor	genre_element	1	Helps in tone-setting and scriptwriting to app...
22	driving pace	pacing	1	Describes a fast-moving narrative structure th...
23	edge of my seat	audience_engagement	1	A key metric for measuring viewer immersion an...
24	edge of my seat	suspense	1	Classic indicator of high tension and effectiv...
21	dialogue delivery	acting	1	Focuses on performance quality and script exec...
25	elaborate stunts	production	1	High-value production elements that attract ac...
26	emotional connection	emotion	1	Emphasizes the need for genuine relationships ...
28	emotionally engaging	audience_engagement	1	Measures the film's ability to connect deeply ...
27	emotional roller-coaster	emotion	1	Reflects a story structure that successfully n...

```
summary_df.to_excel("gemini_crime_movie_keyword_phrases.xlsx", index=False)
```

```
import pandas as pd

file_path = "bottom_10_crime_movies.xlsx"

df = pd.read_excel(file_path)

print(df.columns)
df.head()
```

```
Index(['Unnamed: 0', 'Ratings', 'Reviews', 'movie_name', 'Resenhas', 'genres',
      'Description', 'emotion', 'clean_description', 'clean_reviews'],
      dtype='object')
```

	Unnamed: 0	Ratings	Reviews	movie_name	Resenhas	genres	Description	emotion	clean_description	clean_reviews
0	5631	1	I have no clue why a movie of this quality was...	The Raven	NÃ£o faÃ§o idÃ©ia por que um filme dessa quali...	['Comedy', 'Fantasy', 'Horror']	Add a Plot	anticipation	add a plot	i have no clue why a movie of this quality was...
1	5632	1	I watched this thinking it would be good. Im a...	The Raven	Eu assisti isso pensando que seria bom. Sou um...	['Comedy', 'Fantasy', 'Horror']	Add a Plot	anticipation	add a plot	i watched this thinking it would be good im a ...

```
reviews = df["Reviews"].dropna().astype(str).drop_duplicates().tolist()
```

```
print("Number of reviews:", len(reviews))
```

```
Number of reviews: 185
```

```
class KeywordPhrase(BaseModel):
    phrase: str = Field(
        description="A useful problem-related keyword phrase, preferably 2-4 words."
    )

    category: Literal[
        "genre_element",
        "story_element",
        "pacing",
        "suspense",
        "character",
        "acting",
        "realism",
        "emotion",
        "production",
        "audience_engagement",
        "hybrid_genre"
    ]

    reason: str = Field(
        description="Why this phrase is useful for understanding poor performance."
    )

class KeywordResult(BaseModel):
    phrases: List[KeywordPhrase]
```

```
def extract_useful_phrases_with_gemini(review_batch):

    reviews_text = "\n\n--\n\n".join(review_batch)

    prompt = f"""
You are a professional content analyst for a streaming company.

Task:
```

Extract useful keyword phrases from the following reviews of low-performing crime movies.

Focus on phrases that help explain why these movies performed poorly, especially:

- weak story elements
- poor pacing
- lack of suspense
- confusing plot structure
- weak character development
- poor acting / performance issues
- unrealistic scenes
- weak emotional engagement
- bad production quality
- audience disengagement
- failed genre expectations
- repetitive or predictable crime elements
- unsuccessful hybrid genre elements

Avoid:

- actor names
- movie titles
- country names
- vague negative words
- meaningless common words

Return useful problem-related phrases, preferably 2-4 words each.

Reviews:

```
{reviews_text}
"""
```

```
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt,
    config={
        "temperature": 0.2,
        "response_mime_type": "application/json",
        "response_schema": KeywordResult,
    }
)

return response.parsed
```

```
def chunk_list(data, chunk_size=20):
    for i in range(0, len(data), chunk_size):
        yield data[i:i + chunk_size]
```

```
review_chunks = list(chunk_list(reviews, 20))

print("Number of chunks:", len(review_chunks))
```

Number of chunks: 10

```
all_phrases = []

for i, chunk in enumerate(review_chunks):
    print(f"Processing chunk {i+1}/{len(review_chunks)}...")

    try:
        result = extract_useful_phrases_with_gemini(chunk)

        for item in result.phrases:
            all_phrases.append({
                "phrase": item.phrase.lower().strip(),
                "category": item.category,
                "reason": item.reason
            })

    except Exception as e:
        print(f"Error in chunk {i+1}: {e}")
```

```
Processing chunk 1/10...
Processing chunk 2/10...
Processing chunk 3/10...
Processing chunk 4/10...
Processing chunk 5/10...
Processing chunk 6/10...
Processing chunk 7/10...
Processing chunk 8/10...
Processing chunk 9/10...
Processing chunk 10/10...
```

```
phrases_df = pd.DataFrame(all_phrases)
```